Interview Questions System Design

Generated on February 05, 2026 at 01:29 PM | 43 entries

Q1. How would you design a highly available and scalable URL shortening service like TinyURL or bit.ly from scratch?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: The design involves a few key components. First, an application server handles two main API endpoints: one for creating a short URL (e.g., POST /shorten) and one for redirection (e.g., GET /{shortHash}). For URL generation, we need a hashing strategy. A popular method is to use a 62-character set (a-z, A-Z, 0-9) and a distributed counter (like Zookeeper or a dedicated database sequence) to generate a unique 6-7 character hash. This avoids collisions. The mapping between the short hash and the original long URL is stored in a highly scalable key-value store, like Redis or DynamoDB, which provides low-latency reads. Reads (redirections) are far more common than writes (creations), so the system must be read-heavy. We'd place a Load Balancer in front of the application servers. For redirection (GET /{shortHash}), the server performs a 301 (permanent) redirect to the long URL found in the database. A Content Delivery Network (CDN) and caching at the application layer (caching popular URLs in Redis) are crucial to reduce latency and database load.

Q2. Explain the core concepts of the CAP Theorem and provide real-world examples of systems that prioritize different parts of it.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: The CAP Theorem states that a distributed data store cannot simultaneously guarantee more than two of the following three properties: **C**onsistency (all nodes see the same data at the same time), **A**vailability (every request receives a valid response, even if nodes are down), and **P**artition Tolerance (the system continues to operate despite network failures that split the system into partitions). In modern distributed systems, Partition Tolerance (P) is generally considered non-negotiable. Therefore, designers must choose between Consistency and Availability. **CP (Consistency & Partition Tolerance):** These systems prioritize strong consistency over availability. If a network partition occurs, the system might become unavailable to prevent stale data reads. Examples include financial systems or systems using databases like Google Spanner or CockroachDB. **AP (Availability & Partition Tolerance):** These systems prioritize high availability over strong consistency. During a partition, nodes remain operational, but they might serve stale data. They eventually become consistent once the partition heals. Examples include social media feeds or e-commerce shopping carts, often using databases like Cassandra or DynamoDB.

Q3. Outline the system design for generating and displaying a user's home timeline or news feed on a social media platform like Twitter.

[HARD] | ***Category:*** *System Design & Architecture*

Answer: A timeline or news feed system is complex due to the high read-to-write ratio. A common approach is a hybrid model using 'fan-out-on-write' and 'fan-out-on-read'. **Fan-out-on-write (Push):** When a user (e.g., User A) posts a tweet, the system immediately pushes this tweet into the timeline caches (often a Redis list) of all their followers. When a follower opens their app, their timeline is read directly from this pre-computed cache, making it extremely fast. This works well for users with a few thousand followers. **Fan-out-on-read (Pull):** For celebrities with millions of followers, pushing the tweet to every follower's cache is too slow and resource-intensive (a 'hotkey' problem). Instead, when a normal user loads their timeline, the system (1) fetches their pre-computed feed (from non-celebrities) and (2) separately fetches the recent tweets from all the celebrities they follow. It then merges these two lists in real-time to generate the final timeline. This requires services for posting, timeline generation, a social graph database, and extensive caching.

Q4. What is a Load Balancer, and what are the key differences between Layer 4 (L4) and Layer 7 (L7) load balancing?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: A load balancer is a critical component that distributes incoming network traffic across a group of backend servers (a server farm) to ensure no single server becomes overwhelmed. This improves application availability, reliability, and performance. The primary differences are in the layer of the OSI model they operate on. **L4 (Transport Layer) Load Balancer:** Operates at the transport layer (TCP/UDP). It makes routing decisions based on information from the first few packets, such as source/destination IP addresses and ports. It does not inspect the content of the packets. Because it's simple, it's extremely fast and has low latency. **L7 (Application Layer) Load Balancer:** Operates at the application layer (HTTP/HTTPS). It can inspect the content of the message (e.g., HTTP headers, URL paths, cookies). This allows for much more intelligent, content-aware routing. For example, it can route requests to `/api/video` to video processing servers and `/api/user` to user management servers, or implement sticky sessions based on user cookies.

Q5. Architect a ride-sharing service like Uber or Lyft, focusing on matching riders with nearby drivers and updating driver locations in real-time.

[HARD] | ***Category:*** *System Design & Architecture*

Answer: This system is location-intensive. **Driver Location Tracking:** Drivers' apps continuously send their location (latitude, longitude) via a lightweight protocol (like MQTT or WebSockets) to a Location Service. This service updates the driver's location and status (available, on-trip) in a database. To efficiently find nearby drivers, this data is stored in a database optimized for geospatial queries, using techniques like Geohashing or Quadtrees, which partition the map into a grid. **Rider Request (Matching):** When a rider requests a trip, their app sends the request (riderID, location) to a Matching or Dispatch Service. This service queries the Location Service (e.g., 'find all available drivers within a 5km radius of the rider's Geohash'). It then runs an algorithm to select the 'best' driver based on proximity, rating, and other factors. **Trip Management:** Once matched, a Trip Service manages the state of the ride (accepted, en-route, completed). Asynchronous communication via message queues (like Kafka or RabbitMQ) is used heavily to decouple these services (e.g., notifying the driver, updating the rider app, processing payment).

Q6. How would you design a distributed cache system like Redis or Memcached? What are the key features and design choices?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: A distributed cache pools the RAM of multiple servers to act as one large in-memory cache. Key features include: **Fast Lookups:** A key-value store with O(1) average time complexity. **Distribution:** A sharding mechanism is needed to determine which server holds which key. **Consistent Hashing** is the standard algorithm. It maps both servers and keys to a virtual ring. To find a key, you hash it, find its position on the ring, and walk clockwise to the first server you encounter. This minimizes data redistribution when servers are added or removed. **Eviction Policy:** When the cache is full, a policy is needed to remove old data. The most common is **LRU (Least Recently Used)**, where the least recently accessed item is evicted. **Client Library:** A smart client is often used, which holds the consistent hashing ring information and connects directly to the correct server, avoiding a single point of failure. **Failure Handling:** The system must handle node failures. Typically, this means the data on that node is lost, and the application must fetch it from the database (cache-miss). Some systems (like Redis) offer replication for higher availability.

Q7. What is the difference between a SQL (relational) and a NoSQL (non-relational) database? Provide examples of when you would choose one over the other.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: **SQL Databases** (e.g., PostgreSQL, MySQL) are relational. They store data in structured tables with predefined schemas, enforce relationships between tables, and use SQL (Structured Query Language) for queries. They guarantee **ACID** (Atomicity, Consistency, Isolation, Durability) transactions. **Choose SQL when:** You need strong transactional consistency (e.g., financial systems, e-commerce inventory), your data is highly structured, and you need to perform complex joins and queries. **NoSQL Databases** (e.g., MongoDB, Cassandra, DynamoDB) are non-relational. They come in various types (document, key-value, column-family, graph) and offer flexible schemas, allowing you to store unstructured or semi-structured data. They are designed to scale horizontally and prioritize performance and availability, often providing **BASE** (Basically Available, Soft state, Eventually consistent) guarantees. **Choose NoSQL when:** You have massive datasets that require horizontal scaling (Big Data), your data schema is flexible or evolving rapidly (e.g., user-generated content, IoT data), or you need extremely high write throughput and low latency (e.g., social media feeds, logging).

Q8. How would you design a rate limiter for an API to prevent abuse and ensure fair usage among different users?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: A rate limiter controls the number of requests a user (or IP, or API key) can make in a given time window. **Algorithm Choice:** Common algorithms include **Token Bucket**, **Leaky Bucket**, and **Sliding Window Counter**. The **Sliding Window Counter** is a good hybrid. We use a fast in-memory store like Redis. For each user, we store a count of their requests, timestamped. For a '100 requests per minute' rule, we'd store request counts per second. When a new request comes in, we sum the counts for the last 60 seconds. If the sum is less than 100, we accept the request and increment the counter for the current second. **Architecture:** The rate limiter should be implemented at the edge, either in the **API Gateway** or as a separate middleware. This allows it to reject requests before they hit your core application logic. **Distributed System:** In a distributed environment, all API gateway instances must share the same state. **Redis** is perfect for this, as it provides atomic increment operations (like `INCR`) that prevent race conditions. When a request is rejected, the API should return an **HTTP 429 (Too Many Requests)** status code, and ideally, `Retry-After` headers to inform the client when they can try again.

Q9. Design a notification system that can send push notifications, emails, and SMS messages to millions of users with low latency.

[HARD] | ***Category:*** *System Design & Architecture*

Answer: This system must be scalable, reliable, and support multiple channels. **Components:** 1. **Notification API:** An endpoint for other services to send notification requests (e.g., `POST /notify` with `userID`, `message`, `channels: [push, email]`). 2. **Notification Service:** A central service that receives the request. It fetches user preferences (e.t., 'Do Not Disturb' hours, preferred channels) from a User DB. 3. **Message Queues:** The Notification Service does not send the notification directly. Instead, it enqueues the formatted message into multiple **Message Queues** (like Kafka or RabbitMQ), one for each channel (e.g., `push_queue`, `email_queue`, `sms_queue`). This decouples the services and handles backpressure. 4. **Worker Services:** Independent fleets of workers consume messages from their respective queues. **Email Workers** connect to a third-party provider like SendGrid. **Push Workers** connect to APNS (Apple) and FCM (Google). **SMS Workers** connect to Twilio. 5. **Tracking Service:** A service to handle delivery status callbacks from providers (e.g., 'sent', 'failed') and track user engagement (e.g., 'opened'). This architecture allows each component to be scaled independently.

Q10. What are microservices, and how do they contrast with a monolithic architecture? Discuss the primary pros and cons of adopting a microservice approach.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: A **Monolithic** architecture builds an application as a single, unified, and tightly-coupled unit. All components (e.g., UI, business logic, data access) are developed, deployed, and scaled together. **Microservices** structure an application as a collection of small, independent, and loosely-coupled services, each responsible for a specific business capability (e.g., 'user service', 'payment service', 'product service'). **Pros of Microservices:** **Independent Deployment:** Services can be updated and deployed individually without affecting the entire application. **Technology Diversity:** Each service can use the best technology stack for its specific job. **Fault Isolation:** A failure in one service (e.g., 'recommendation service') won't crash the entire application (e.g., 'checkout service'). **Scalability:** Services can be scaled independently (e.g., scale the 'video streaming' service without scaling the 'user login' service). **Cons of Microservices:** **Operational Complexity:** Managing, monitoring, and debugging many moving parts is much harder. **Network Latency:** Services communicate over the network, which is slower than in-process calls in a monolith. **Distributed Data:** Maintaining data consistency across multiple services is challenging, often requiring complex patterns like Sagas.

Q11. Design a search autocomplete or typeahead suggestion system, like the one used in the Google search bar, for a large e-commerce website.

[HARD] | ***Category:*** *System Design & Architecture*

Answer: This system must be extremely fast, providing suggestions in milliseconds. **Data Structure:** The core of this system is a **Trie** (or prefix tree). Each node in the Trie represents a character, and a path from the root to a node forms a prefix. We can store the top 10 most frequent search queries that share that prefix at each node. **Data Source & Ranking:** The Trie is built from historical search query data. Queries are ranked by frequency or popularity. To make suggestions relevant (e.t., 'iPhone 15' is more popular than 'iPhone 3'), we can use a weighted average or decay factor, giving more weight to recent searches. **System Architecture:** 1. **Trie Construction:** A batch job (e.g., a Spark job) runs daily to process search logs, build a new Trie, and rank the suggestions. 2. **Trie Service:** The constructed Trie is loaded into memory on a fleet of dedicated 'Trie Servers'. Since the Trie can be very large, it might be sharded (e.g., by the first character of the prefix). 3. **Application Layer:** When a user types a character, the request hits an application server. This server queries the Trie Service for that prefix (e.g., 'iph'), retrieves the pre-computed list of top suggestions, and returns it. **Caching:** We can heavily cache the results for very common prefixes (e.g., 'i', 'ip', 'iph') in a distributed cache like Redis or at the CDN layer to reduce load on the Trie Servers.

Q12. Explain the purpose of a Content Delivery Network (CDN) and how it helps improve the performance and availability of a website.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: A CDN (Content Delivery Network) is a geographically distributed network of proxy servers. Its primary purpose is to deliver static content (like images, videos, JavaScript files, CSS) to users based on their geographic location. **How it works:** When a user requests a file (e.g., `logo.png`), the request is routed to the nearest CDN 'edge server' instead of the application's origin server. If the edge server has the file cached, it serves it directly, which is very fast. If it's a 'cache miss', the edge server fetches the file from the origin server, caches it for future requests, and then serves it to the user. **Benefits:** 1. **Reduced Latency:** By serving content from a server physically closer to the user, data transfer times are significantly reduced, making the website feel faster. 2. **Reduced Origin Server Load:** The CDN offloads the traffic for static assets, freeing up the origin server's resources (CPU, bandwidth) to focus on dynamic content and business logic. 3. **Higher Availability & Fault Tolerance:** If the origin server goes down, the CDN can often continue to serve the cached content, providing a degree of graceful degradation. 4. **Security:** Many CDNs also provide security features like DDoS mitigation and Web Application Firewalls (WAF).

Q13. What is database indexing, and how does it work? What are the trade-offs involved in adding indexes to a database?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: An **index** is a data structure, most commonly a **B-Tree**, that improves the speed of data retrieval operations on a database table. It works like the index in a book: instead of scanning the entire book (a 'full table scan') to find a topic, you look it up in the index, which points directly to the correct page (the data block on disk). When you add an index to a column (e.g., `email`), the database maintains a sorted B-Tree of all email values, with pointers to the corresponding rows. When you run `WHERE email = '...'`, the database can use this B-Tree to find the row in O(log N) time, which is vastly faster than a full scan's O(N) time. **Trade-offs:** **Reads:** Indexes dramatically **speed up `SELECT` queries** (especially with `WHERE`, `JOIN`, and `ORDER BY` clauses). **Writes:** Indexes **slow down `INSERT`, `UPDATE`, and `DELETE` operations**. This is because every time you modify data, the database must also update every index that includes that data to keep it sorted. **Storage:** Each index consumes additional disk space. Therefore, you should not index every column. You should only index columns that are frequently used in query `WHERE` clauses to filter data.

Q14. Explain the difference between horizontal and vertical scaling, and discuss when you would apply each.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: **Vertical Scaling (Scaling Up):** This involves increasing the resources of a single server. This means adding more CPU, RAM, or faster storage (SSDs) to an existing machine. **Pros:** It's simple to implement (no code changes, just a hardware upgrade) and can be very effective for a time. **Cons:** There is a physical limit to how much you can upgrade a single machine, and it can become prohibitively expensive. It also creates a single point of failure (SPOF); if that one massive server fails, your entire application goes down. **Horizontal Scaling (Scaling Out):** This involves adding more servers to your application and distributing the load across them, usually with a load balancer. **Pros:** It offers virtually limitless scalability, is more cost-effective (using commodity hardware), and provides high availability (if one server fails, the load balancer simply routes traffic to the others). **Cons:** It's more complex to implement. Your application must be designed to be stateless, and you need to manage distributed state, data consistency, and service discovery. **When to use:** You often start with vertical scaling as it's the easiest way to handle initial growth. You move to horizontal scaling when you hit the limits of vertical scaling, or more importantly, when you need to build a fault-tolerant and highly available system from the start.

Q15. Explain the concept of 'eventual consistency' and how it differs from 'strong consistency' in distributed databases. Provide a real-world example.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: **Strong Consistency:** This is the traditional database model. It guarantees that after an update operation completes, any subsequent read operation from any client will return the updated value. All nodes in the cluster agree on the data's state *before* confirming the write. This is achieved using protocols like two-phase commit, but it comes at the cost of higher latency. **Eventual Consistency:** This model guarantees that if no new updates are made to a given data item, all accesses to that item will *eventually* return the last updated value. There is a (usually brief) window of time where different nodes may return different, older values. The system eventually converges to a consistent state. **Example:** A social media 'like' count. When you 'like' a post, you see the count increment immediately. However, your friend in another country might see the old count for a few more seconds. The system prioritizes **availability** (letting you 'like' the post) over strong consistency (ensuring everyone sees the new count instantly). This is acceptable because the exact 'like' count not being globally synchronized for a few seconds has no critical impact. In contrast, a bank transfer *must* use strong consistency.

Q16. What is the 'fan-out' pattern in system design, and how is it used in designing a social media feed?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: The 'fan-out' pattern describes how a single piece of information is distributed to multiple consumers. In a social media context, it's about how one user's post gets delivered to all their followers. There are two main approaches: **1. Fan-out-on-write (Push Model):** When a user posts, the system *immediately* identifies all their followers (from a social graph database) and 'pushes' that post (or a reference to it) into a pre-computed 'timeline cache' (e.g., a Redis List) for *each* follower. When followers load their timeline, they are just reading this pre-built list, which is extremely fast. This is the preferred method for most users. **2. Fan-out-on-read (Pull Model):** When a user loads their timeline, the system *at that moment* 'pulls' the recent posts from *everyone* they follow, merges them, sorts them, and then displays the result. This is computationally expensive and slow at read-time, so it's generally avoided. However, it's used as a hybrid solution for 'celebrity' users, as 'fan-out-on-write' would be too slow if a celebrity with 50 million followers posts.

Q17. Describe the differences between monolithic, Service-Oriented Architecture (SOA), and microservices architectures.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: **Monolithic:** The entire application is built, deployed, and scaled as a single, large, tightly-coupled unit. All logic (UI, business, data access) resides in one codebase. It's simple to develop and deploy initially but becomes difficult to maintain, scale, and update as it grows. **Service-Oriented Architecture (SOA):** This was an evolution from monoliths. It breaks the application into larger, coarse-grained services (e.g., 'Finance', 'HR', 'Billing'). These services often communicate through a central 'Enterprise Service Bus' (ESB), which acts as a smart message router, handling transformations and routing. The services are often still large and can be slow to change. **Microservices:** This is a more granular form of SOA. It breaks the application into very small, independent services, each focused on a single business capability (e.g., 'UserService', 'PaymentService'). Services are loosely coupled and communicate over lightweight, 'dumb' pipes like simple HTTP APIs or message queues. Each service can be developed, deployed, and scaled independently, offering maximum flexibility and fault isolation.

Q18. How would you design a system to find the top K most frequently viewed products on an e-commerce site in real-time?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: This is a classic 'Top K' streaming problem. We need a system that can process a high volume of 'product_view' events and maintain an accurate, real-time count. **1. Ingestion:** Use **Kafka** to ingest all `product_view` events. Each event contains `{ 'product_ID': '123', 'timestamp': ... }`. **2. Stream Processing:** Use a stream processor like **Apache Flink** or **Spark Streaming**. This processor will consume the Kafka stream and count the events in a 'sliding window' (e.g., the last 5 minutes). **3. The 'Top K' Algorithm:** You cannot store counts for all products in memory. Instead, you use a probabilistic data structure. A combination of **Count-Min Sketch** and a **Min-Heap** is a standard solution. **Count-Min Sketch:** A probabilistic data structure that estimates the frequency of events in a stream using a small, fixed amount of memory. It can overestimate but never underestimates. **Min-Heap:** A heap data structure of size K. As the stream processor gets product counts from the Count-Min Sketch, it compares them to the smallest item in the Min-Heap. If the new product's count is larger, it replaces the smallest item in the heap. **4. Serving:** The stream processor continuously updates the Min-Heap. The final Top K list from this heap is periodically pushed to a **Redis** cache. A dashboard or API can then read from this Redis cache to display the real-time Top K products with very low latency.

Q19. Explain what 'consistent hashing' is and why it is a crucial concept for distributed caching and sharding.

[HARD] | ***Category:*** *System Design & Architecture*

Answer: Consistent hashing is an algorithm used to distribute data (keys) across a cluster of servers (nodes). **The Problem with Simple Hashing:** A simple approach like `server = hash(key) % N` (where N is the number of servers) is disastrous. If you add or remove a server, N changes, and nearly *all* keys will be re-mapped to different servers. This would cause a 'thundering herd' on your database, as the cache becomes almost completely ineffective. **How Consistent Hashing Works:** 1. It maps all servers and all keys onto a virtual 'ring' (e.g., with values from 0 to 2^32). 2. To find where a key belongs, you hash the key to get its position on the ring. 3. You then move 'clockwise' around the ring until you find the first server. That server is responsible for that key. **The Benefit:** When a server is added, it takes a place on the ring. It only takes responsibility for the keys that fall between it and the next server counter-clockwise. When a server is removed, its keys are redistributed *only* to the next server clockwise. In both cases, only a small fraction (1/N) of the keys are moved, not the entire dataset. This makes it ideal for distributed caches (like Memcached) and databases (like DynamoDB) where servers can join and leave the cluster dynamically.

Q20. What is the role of an API Gateway in a microservices architecture? What key features does it provide?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: An API Gateway is a server that acts as the single entry point for all client requests into an application's backend. In a microservices architecture, instead of clients (like mobile or web apps) calling dozens of different microservices directly, they make a single request to the API Gateway. The Gateway then routes the request to the appropriate downstream service(s). **Key Features:** 1. **Request Routing:** It's a reverse proxy that maps incoming requests (e.g., `/api/v1/profile`) to the correct internal service (e.g., `user-service:8080`). 2. **Authentication & Authorization:** It centralizes security. It can verify authentication tokens (like JWTs) or API keys before forwarding a request, so individual services don't have to. 3. **Rate Limiting & Throttling:** It protects backend services from abuse and overload by enforcing usage policies (e.g., 100 requests/minute per user). 4. **Aggregation (Fan-out):** It can receive one client request and make multiple calls to different microservices, then aggregate the results and send a single response back to the client. 5. **Protocol Translation:** It can translate protocols, e.g., accepting an external REST/HTTP request and converting it to gRPC for internal communication. 6. **Logging & Monitoring:** It's a central place to log all incoming traffic and gather metrics.

Q21. Explain the difference between a message queue (like RabbitMQ) and a log-based message broker (like Apache Kafka).

[HARD] | ***Category:*** *System Design & Architecture*

Answer: **Message Queue (e.g., RabbitMQ):** This is a traditional 'smart broker, dumb consumer' model. It uses a queue data structure (FIFO). When a consumer reads a message, the message is *removed* from the queue. This is ideal for **task distribution**, where you want multiple workers to consume tasks from a queue, and each task should only be processed *once*. The broker manages the state of which consumer has which message. **Log-Based Broker (e.g., Apache Kafka):** This is a 'dumb broker, smart consumer' model. Kafka doesn't use a queue; it uses a persistent, append-only **commit log**. Messages are *never* removed from the log (until they expire after a retention period, e.g., 7 days). Multiple consumers can read from the same log (called a 'topic') independently. Each consumer group tracks its own 'offset' (its position in the log). This makes Kafka ideal for **data streaming** and **event sourcing**, where you might want multiple, different services (e.g., an analytics service, a monitoring service, and an archive service) to all read the same stream of events at their own pace.

Q22. Design a 'pastebin' service where users can paste text and get a unique URL. What are the key considerations?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: This is similar to a URL shortener but with a different payload. **Functional Requirements:** 1. Users can paste a block of text. 2. The system generates a unique URL (e.g., `paste.bin/AbC12xY`). 3. Users can access the text by visiting the URL. 4. Pastes can have an expiration time. **Architecture:** 1. **Application Servers:** Behind a load balancer, handling two main endpoints: `POST /create` (takes `content` and `expiration` as input) and `GET /{paste_id}`. 2. **Data Storage:** We have two types of data. **Metadata:** The mapping from `paste_id` to the location of the content, plus other info like `creation_time`, `expiration_time`, etc. This can be stored in a **SQL Database** (sharded by `paste_id`) or a **NoSQL Key-Value store** (like DynamoDB). **Content (The 'Paste'):** The text content itself can be large. Storing this large text blob directly in the primary database is inefficient. It's better to store it in an **Object Storage** system like **Amazon S3** or in a separate, dedicated NoSQL database (like Cassandra) optimized for large blobs. The metadata entry would then just contain a pointer to the S3 object. 3. **Paste ID Generation:** We can use the same 62-character hash strategy as a URL shortener. 4. **Expiration:** A background worker job (e.g., a Cron job) needs to run periodically, scan the metadata database for expired pastes, and delete the corresponding content from S3 and the metadata from the database.

Q23. Describe the 'Saga' pattern for managing data consistency across microservices. Why is it used over two-phase commits?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: In a microservices architecture, a single 'transaction' (like 'placing an order') might involve multiple services: 'Order Service', 'Payment Service', and 'Inventory Service'. We cannot use a traditional database 'ACID' transaction across these separate services. **Two-Phase Commit (2PC)** is a protocol to achieve this, but it's rarely used because it's a blocking, synchronous protocol. It requires a central 'coordinator', and if *any* service (or the coordinator) fails, the entire transaction is locked, leading to poor availability. **The Saga Pattern** is an alternative that manages consistency using a sequence of *local* transactions. A saga is a sequence of events. Each service performs its own local transaction and then publishes an event to notify the next service in the chain. **Types of Sagas:** 1. **Choreography (Event-Based):** Services communicate by publishing and subscribing to events. 'Order Service' creates an order and publishes an `ORDER_CREATED` event. 'Payment Service' listens for this, processes payment, and publishes `PAYMENT_SUCCESSFUL`. 'Inventory Service' listens for that and updates stock. 2. **Orchestration (Command-Based):** A central 'Orchestrator' service manages the entire process. It sends commands to each service (e.g., 'Debit Customer', 'Update Stock') and waits for replies. **Compensation:** The key to Sagas is handling failures. If a step fails (e.g., 'Payment Service' fails), the Saga must execute 'compensating transactions' to undo the preceding work (e.g., 'Order Service' must run a `CANCEL_ORDER` transaction). This is an 'eventually consistent' model that prioritizes availability.

Q24. How does a video streaming service like Netflix or YouTube deliver high-definition video to millions of users globally with low latency?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: This is achieved through two main strategies: a massive **Content Delivery Network (CDN)** and **Adaptive Bitrate Streaming (ABR)**. **1. CDN:** Netflix doesn't stream videos from one central datacenter. It has its own CDN (called Open Connect) where servers (OCAs) are placed all over the world, often directly inside ISP datacenters. When you want to watch a video, the content is streamed from an OCA that is physically very close to you, which dramatically reduces latency and network congestion. **2. Adaptive Bitrate Streaming (ABR):** When a video is uploaded, it's not stored as one file. It's transcoded into many different versions (e.g., 480p, 720p, 1080p, 4K) and also broken into small, 2-10 second 'chunks'. Your video player (in your browser or app) starts by requesting a lower-quality chunk. It then continuously monitors your network bandwidth. If your connection is fast, it will start requesting the higher-quality (e.g., 1080p) chunks. If your connection slows down, it will seamlessly switch to requesting the lower-quality (e.g., 480p) chunks. This prevents buffering and ensures the video keeps playing even on a fluctuating network.

Q25. What is the 'Circuit Breaker' pattern, and why is it essential in a microservices architecture?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: The Circuit Breaker pattern is a resilience strategy that prevents an application from repeatedly trying to call a service that is known to be failing. In a microservices architecture, services often depend on each other. If one service (e.g., 'Review Service') slows down or fails, any service that calls it (e.g., 'Product Page Service') will also slow down, holding onto connections and threads while it waits for a timeout. This can cause a 'cascading failure', bringing down the entire system. **How it works:** The Circuit Breaker acts as a state machine: **Closed:** This is the normal state. All requests are allowed to pass through to the failing service. A failure counter is maintained. **Open:** If the number of failures (e.g., timeouts) exceeds a threshold in a given time, the circuit 'trips' and moves to the 'Open' state. In this state, all calls to the failing service are *immediately* rejected without even trying to connect. This gives the failing service time to recover. **Half-Open:** After a specified 'cooldown' period, the circuit moves to 'Half-Open'. It allows a *single* test request to go through. If that request succeeds, the circuit moves back to 'Closed'. If it fails, the circuit goes back to 'Open', and the cooldown timer restarts. This prevents a thundering herd from overwhelming a service that is just starting to recover.

Q26. Design a 'web crawler' that can scan the entire internet. What are the main components and challenges?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: A web crawler is a distributed system that browses the web to index its content. **Main Components:** 1. **Seed URLs:** A starting list of URLs to begin crawling (e.g., a few popular domains). 2. **URL Frontier:** A massive queue that stores all the URLs that have been discovered but not yet crawled. This needs to be a distributed, prioritized queue, managing billions of URLs. 3. **Crawler Workers:** A large fleet of servers that pull URLs from the Frontier. Each worker: a. Fetches the HTML content of the page. b. Parses the HTML to extract all `<a>` (link) tags. c. Adds these newly discovered URLs back into the Frontier. d. Saves the fetched page content to a 'Content Store'. 4. **Content Store:** A distributed object store (like S3) or a Bigtable-style database to store the raw HTML of all crawled pages. 5. **URL Seen' Filter:** A very large, probabilistic data structure (like a **Bloom Filter**) used to check if a URL has *already* been added to the Frontier or crawled. This is crucial for breaking out of 'spider traps' (infinite loops of links) and avoiding duplicate work. **Key Challenges:** 1. **Scale:** The web is massive. The system must be highly distributed. 2. **Politeness:** Crawlers must obey the `robots.txt` file of each website (which specifies which paths are off-limits) and must 'rate limit' their requests to avoid overloading any single website. 3. **Duplicate Content:** Many URLs point to the same content. The system needs a way to 'canonicalize' URLs and detect duplicate pages.

Q27. What are 'push' vs. 'pull' models of communication in distributed systems? Give an example of each.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: This describes the direction of data flow initiation between a 'producer' (which creates data) and a 'consumer' (which needs the data). **Pull Model (Client-Initiated):** The consumer is responsible for *asking* for the data. It 'pulls' data from the producer. **Example: HTTP / REST APIs.** Your web browser (the consumer) 'pulls' the content of a web page by sending an HTTP `GET` request to the web server (the producer). The server only sends data when it is requested. **Pros:** Simple, stateless, and the consumer controls the rate of consumption. **Cons:** Can be inefficient if the consumer needs data immediately. The consumer must 'poll' the server (ask repeatedly, 'Do you have new data yet?'), which creates a lot of useless traffic. **Push Model (Server-Initiated):** The producer is responsible for *sending* the new data to the consumer as soon as it's available. **Example: WebSockets or Push Notifications.** In a chat app (using WebSockets), the server (producer) 'pushes' a new message to your phone (consumer) the instant it's received from your friend. You don't have to keep polling the server. **Pros:** Very low latency and highly efficient for real-time updates. **Cons:** More complex, as it requires a persistent connection and state management on the server (to know which consumers are connected).

Q28. What is the difference between caching strategies like 'Cache-Aside', 'Write-Through', and 'Write-Back'?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: These patterns describe how a cache and a primary database (DB) are kept in sync. **1. Cache-Aside (Lazy Loading):** This is the most common pattern. **Read:** The application first checks the cache. If it's a 'cache miss' (data not found), the application reads the data from the DB, *then* writes that data into the cache, and finally returns it. **Write:** The application writes the new data *directly* to the DB. It can *optionally* invalidate (delete) the corresponding key in the cache. **Pros:** Resilient. If the cache fails, the app still works (just slower, as it hits the DB). **Cons:** 'Cache miss' results in three steps (read cache, read DB, write cache), which is a slight latency hit. Data can become stale if another service writes to the DB without invalidating the cache. **2. Write-Through:** **Read:** Same as Cache-Aside. **Write:** The application writes the new data *to the cache first*. The cache itself is then responsible for synchronously writing that data to the DB. **Pros:** Data in the cache is never stale. **Cons:** Writes are slower because they have to complete two operations (write to cache + write to DB) before returning success. If the cache fails, writes fail. **3. Write-Back (Write-Behind):** **Read:** Same as Cache-Aside. **Write:** The application writes data *only* to the cache, which is extremely fast. The cache then asynchronously writes the data to the DB in the background after a delay. **Pros:** Extremely fast writes and high write throughput. **Cons:** High risk of data loss. If the cache server crashes before the data is 'flushed' to the DB, that data is permanently lost. This is only used when performance is critical and some data loss is acceptable.

Q29. Explain the 'Leader-Follower' (or 'Master-Slave') replication model in databases. What are its pros and cons?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: Leader-Follower is a common architecture for database replication. **How it works:** 1. **Leader (Master):** One database server is designated as the 'Leader'. It is responsible for handling *all write operations* (`INSERT`, `UPDATE`, `DELETE`). 2. **Followers (Slaves):** One or more 'Follower' servers are created. 3. **Replication Log:** The Leader records all its data changes in a replication log (like a binary log). 4. **Propagation:** The Followers connect to the Leader, read this log, and apply the same changes to their own local copy of the data. This process can be **Synchronous** (Leader waits for Followers to confirm) or **Asynchronous** (Leader writes and moves on, Followers catch up eventually). **Pros:** **Read Scalability:** Read queries (which are often 90% of traffic) can be distributed across all the Follower nodes, taking the load off the Leader. **High Availability:** If the Leader server fails, one of the Followers (which has an up-to-date copy of the data) can be 'promoted' to become the new Leader, allowing the system to recover. **Cons:** **Write Bottleneck:** All writes must still go through the single Leader, so write scalability is limited. **Replication Lag:** In asynchronous replication (the most common type), there is a delay (from milliseconds to seconds) before the changes are copied to the Followers. This means reads from a Follower might return 'stale' or old data, leading to eventual consistency.

Q30. What is 'event sourcing' and how does it differ from traditional state-based persistence?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: **Traditional (State-Based) Persistence:** This is what most applications do. When something changes (e.g., a user updates their shipping address), the application *overwrites* the old data in the database with the new data. The 'state' (the current address) is stored, but the *history* of how it got to that state is lost. **Event Sourcing:** In this pattern, you *never* delete or overwrite data. Instead, you store every single change as an immutable 'event' in an append-only log. **Example:** 1. `UserCreated (userID: 123, name: 'Alice')` 2. `AddressAdded (userID: 123, address: '123 Main St')` 3. `AddressChanged (userID: 123, newAddress: '456 Park Ave')` The *current state* of the user is not stored directly. To get the current state, you 'replay' all the events for that user from the beginning. **Pros:** **Full Audit Log:** You have a perfect, immutable history of every change ever made, which is invaluable for debugging and analytics. **Temporal Queries:** You can reconstruct the state of the system at *any point in time* (e.g., 'What was this user's address last Tuesday?'). **Cons:** **Query Complexity:** Getting the 'current state' is more complex, as it requires replaying events. This is often solved using **CQRS (Command Query Responsibility Segregation)**, where a separate, 'materialized view' (a read-optimized database) is built by listening to the event stream.

Q31. What is the difference between 'long polling', 'WebSockets', and 'Server-Sent Events (SSE)'? When would you use each?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: These are three techniques for a server to send data to a client in real-time, overcoming the limitations of the classic HTTP request-response model. **1. Long Polling:** The client makes a normal HTTP request to the server, but the server *holds the connection open* and doesn't respond immediately. It only sends a response when it *has* new data. As soon as the client receives the response, it immediately makes *another* long-polling request. **Use:** A simple, legacy way to get 'push-like' behavior without new protocols. It has high overhead due to repeated connections. **2. WebSockets:** This provides a **bi-directional**, stateful, full-duplex communication channel over a single, long-lived TCP connection. Both the client and the server can send data to each other at any time, independently. **Use:** This is the most powerful and is used for true real-time, two-way communication. **Examples: Chat applications** (like Slack or WhatsApp), multi-player online games, and collaborative editors (like Google Docs). **3. Server-Sent Events (SSE):** This provides a **one-way** (server-to-client) stream of data over a single, long-lived HTTP connection. The server can push data to the client, but the client cannot send data to the server (except for the initial connection). **Use:** This is perfect when you only need updates *from* the server. **Examples: Stock tickers**, live sports score updates, or a news feed that pushes new headlines. It's simpler and lighter than WebSockets.

Q32. What is 'polyglot persistence', and why would a microservices-based architecture adopt this approach?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: 'Polyglot persistence' is the idea of using multiple different database technologies ('poly' = many, 'glot' = tongue) within a single application, choosing the *best* database for each specific job, rather than using a 'one-size-fits-all' relational database. **Why it's used in Microservices:** A microservices architecture naturally supports this. Since each service is independent and owns its own data, each service team is free to choose the database technology that best fits its specific needs. **Example:** In a single e-commerce application: 1. The **User Service** (which handles login, profiles) might use a **Relational DB (like PostgreSQL)** because it needs strong consistency and can manage user relations. 2. The **Product Catalog Service** (which stores flexible product attributes) might use a **Document DB (like MongoDB)** because its schema-less nature is perfect for handling products with varying attributes. 3. The **Shopping Cart Service** (which needs fast, temporary key-value storage) might use an **In-Memory Cache (like Redis)**. 4. The **Social Graph Service** ('who follows whom' for reviews) might use a **Graph DB (like Neo4j)** to efficiently query complex relationships. This approach optimizes the performance and scalability of each individual service, but it increases operational complexity (you now have to manage 4 different database systems).

Q33. What are the potential problems with a microservices architecture, and how would you address them?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: While popular, microservices introduce significant challenges. **1. Network Latency & Reliability:** Services communicate over the network, which is inherently slower and less reliable than in-process calls in a monolith. **Solution:** Use efficient protocols (like gRPC), design for failure using patterns like 'retries' (with exponential backoff) and 'circuit breakers' to prevent cascading failures. **2. Distributed Data Consistency:** You can't use a single ACID transaction across services. **Solution:** Use the **Saga Pattern**. This is an eventually consistent model where each service performs a local transaction and then publishes an event. If a step fails, the saga triggers 'compensating transactions' to undo the previous work. **3. Operational Complexity (Observability):** Debugging a request that spans 10 different services is extremely difficult. **Solution:** You *must* have centralized **logging** (e.t., ELK stack), **metrics** (e.g., Prometheus), and **distributed tracing** (e.g., Jaeger or OpenTelemetry). Tracing assigns a unique 'trace ID' to each request at the API Gateway, which is then passed to every downstream service, allowing you to visualize the entire request's journey. **4. Service Discovery:** When services are ephemeral and scale up and down, how do they find each other's IP addresses? **Solution:** Use a **Service Registry** (like Consul or Eureka). When a service starts, it registers its location. When another service wants to call it, it queries the registry to find its current address.

Q34. How would you design a 'trending' feature, such as the top 10 most popular hashtags on Twitter in the last hour?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: This is a streaming 'Top K' problem with a 'sliding window'. **1. Ingestion:** A massive stream of events (e.g., all tweets) must be ingested. This should go into a **Kafka** topic. **2. Stream Processing:** A stream processor (like **Apache Flink** or **Spark Streaming**) consumes this stream. As tweets arrive, it parses them to extract the hashtags. **3. Counting in a Sliding Window:** We need the counts for the 'last hour'. The processor can't store all hashtags. It uses a **Sliding Window Counter**. The processor maintains counts in 1-minute 'buckets'. The 'last hour' count is the sum of the last 60 buckets. When a new minute starts, a new bucket is created, and the oldest bucket (from 61 minutes ago) is dropped. **4. 'Top K' Algorithm:** Storing counts for *all* hashtags is impossible. We use a probabilistic algorithm like **Count-Min Sketch** to estimate the frequency of each hashtag in a memory-efficient way. We combine this with a **Min-Heap** of size 10 (for 'Top 10'). As the stream processor gets a new count for a hashtag, it compares it to the smallest item in the heap. If it's larger, it's added to the heap, and the smallest item is removed. **5. Serving Layer:** The stream processor periodically (e.g., every 5 seconds) flushes its current 'Top 10' list from the Min-Heap into a **Redis** cache. The application API that serves the 'Trending' list simply reads from this Redis cache, making it extremely fast.

Q35. What is database 'normalization' and 'denormalization'? What are the trade-offs between them in system design?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: **Normalization:** This is the process of organizing data in a relational database to reduce **data redundancy** and improve **data integrity**. This is done by splitting data into multiple tables that are related by keys. For example, instead of storing a user's name and address in every 'Orders' table row, you store a `user_ID`. The user's address is stored only *once* in a separate 'Users' table. **Pros:** High data integrity (address is updated in one place), less storage space. **Cons:** Queries are slower. To get the user's address for an order, you must perform a `JOIN` operation, which is computationally expensive. **Denormalization:** This is the opposite process. It involves intentionally adding **redundant data** to tables to improve query performance by *reducing* the number of `JOINs`. Using the same example, you *would* copy the user's shipping address and store it directly in the 'Orders' table. **Pros:** Queries are much faster (a simple `SELECT` from one table, no `JOIN` needed). This is critical for read-heavy analytics systems. **Cons:** Lower data integrity and more storage. If the user updates their 'default' address, it *does not* update the addresses on their past orders (which is often the desired business logic), and data is duplicated. **Trade-off:** You normalize for write-heavy, transactional (OLTP) systems where data integrity is paramount. You denormalize for read-heavy, analytical (OLAP) systems or data warehouses where query speed is the priority.

Q36. Explain the concept of 'sharding' or 'horizontal partitioning' in detail. What are common sharding strategies?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: Sharding is a database scaling technique that splits a very large database horizontally (by rows) into smaller, faster, more manageable pieces called 'shards'. Each shard is its own independent database, often on its own server. This is used to achieve **write scalability** when a single server can no longer handle the write load or data volume. **Sharding Strategies (How to decide which shard a row goes to):** 1. **Range-Based Sharding:** Data is sharded based on a range of values. For example, a 'Users' table could be sharded by zip code (e.g., Shard 1: 00000-19999, Shard 2: 20000-39999). **Pro:** Easy to implement. **Con:** Can lead to 'hotspots'. If all your users are in one zip code range, that shard will be overwhelmed. 2. **Hash-Based Sharding:** Data is sharded based on the *hash* of a 'shard key'. For example, you take the `user_ID`, calculate its hash, and then run a modulo operator (`hash(user_ID) % num_shards`) to find its shard. **Pro:** This distributes data very evenly, avoiding hotspots. **Con:** Range queries are impossible (e.g., 'get all users with IDs 100-200'). **3. Geo-Sharding:** A common strategy for global applications. Data is sharded based on a user's location (e.g., `country`). All European user data lives on servers in Europe, all US data lives on US servers. **Pro:** This provides low latency for users and helps with data privacy regulations (like GDPR).

Q37. What is the 'Single Point of Failure' (SPOF) problem, and how do you design systems to avoid it?

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: A Single Point of Failure (SPOF) is any component in a system that, if it fails, will cause the entire system to stop working. In a simple architecture, this could be the single web server, the single load balancer, or the single database. **How to Avoid SPOF (The core principle is Redundancy):** 1. **Load Balancers:** You never have just one. You run two or more in an **Active-Passive** or **Active-Active** cluster. If the active load balancer fails, the passive one (which shares a virtual IP) takes over instantly. 2. **Web/Application Servers:** This is the easiest. You run multiple servers (a 'cluster') *behind* the load balancer. If one server fails, the load balancer's health checks will detect it and stop sending traffic to it. This is horizontal scaling. 3. **Databases:** This is the hardest. You use **Replication**. You have a **Leader (Master)** database and one or more **Follower (Replica)** databases that get a real-time copy of the data. If the Leader fails, an automated 'failover' process promotes one of the Followers to become the new Leader. 4. **Datacenters:** For true high availability, you design for 'regional' or 'datacenter' failure. You run a complete, redundant copy of your entire application in a separate geographic region (e.g., 'us-east-1' and 'us-west-2' on AWS), with DNS-level load balancing (like AWS Route 53) to route traffic between them.

Q38. What are optimistic and pessimistic concurrency control (locking)? When would you use one over the other?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: Concurrency control is how databases manage simultaneous access to the same piece of data. **1. Pessimistic Concurrency Control (Locking):** This strategy assumes that conflicts *will* happen. When a transaction wants to read data, it acquires a **lock** on it (e.g., a 'read lock' or an 'exclusive write lock'). This prevents *any other* transaction from modifying (or even reading) that data until the first transaction is finished and releases the lock. **Use:** This is used in systems where data integrity is absolutely critical and conflicts are common. **Example: A banking system.** When you transfer money, the system *must* lock your account row to prevent another transaction (like an ATM withdrawal) from reading the old balance. It prioritizes consistency over performance. **2. Optimistic Concurrency Control (OCC):** This strategy assumes that conflicts are *rare*. Transactions *do not* take locks. Instead, each transaction reads the data along with a 'version' number (or timestamp). When the transaction is ready to commit its write, it checks if the data's *current* version in the database is still the *same* as the version it first read. **If yes (no conflict):** The write is committed, and the version number is incremented. **If no (conflict!):** The database rejects the transaction. The application must then 'retry' the entire transaction (read the new data, re-apply the logic, and try to commit again). **Use:** This is used in high-throughput systems where conflicts are rare, like a wiki or a blog. It has much higher performance because it avoids the overhead of locking.

Q39. Design a 'like' button system for a social media site. What are the key architectural considerations for handling millions of likes?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: A 'like' system is extremely write-heavy, but it does not require strong, immediate consistency. **Functional Requirements:** 1. A user can 'like' a post. 2. A user can 'unlike' a post. 3. The system must display the total 'like' count for a post. 4. The system must show if the *current* user has liked the post. **Architecture:** **1. Data Model:** A NoSQL database is ideal. We can use a wide-column store like **Cassandra** or **DynamoDB**. The primary key should be the `post_ID`. This allows all data related to a post to be on a single partition. We can have one table for counts and another for user likes: * `PostLikesCount`: `(post_ID, like_count)` * `UserLikes`: `(post_ID, user_ID)` (This tells us *who* liked it) **2. Handling the 'Like' Click (The Write Path):** This needs to be fast. When a user clicks 'like', the request hits an API server. The server *does not* write to the database immediately. Instead, it: 1. Publishes a 'like' event to a **Kafka** topic. 2. Immediately returns a '200 OK' to the client, which updates the UI to show the 'liked' state. **3. Background Processing:** A separate fleet of **Workers** consumes from the Kafka topic. These workers are responsible for the database writes: they increment the `like_count` in the `PostLikesCount` table and add the `user_ID` to the `UserLikes` table. This is **eventual consistency**. The count might be a few seconds out of date, which is an acceptable trade-off for massive write scalability. **4. The Read Path:** When a user loads a post, the API fetches the `like_count` from `PostLikesCount` and also does a quick lookup (e.g., `SELECT user_ID FROM UserLikes WHERE post_ID = ? AND user_ID = ?`) to see if the current user's ID is in the list, which determines if the 'like' button should be 'blue' or 'grey'. Aggressive caching (e.g., in Redis) is used for the `like_count` of popular posts.

Q40. Explain the concept of 'idempotency' in API design and why it is crucial for building reliable distributed systems.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: An operation is **idempotent** if it can be performed multiple times, but the result is the *same* as if it had been performed only once. In other words, multiple identical requests have the same effect as a single one. **Why is this crucial?** In a distributed system, network failures are a fact of life. A client (e.g., a mobile app) might send a `POST /pay` request. The server might process the payment, but the '200 OK' response gets lost on the network. The client, not receiving a response, will assume the request failed and *retry* it. **If the `/pay` endpoint is *not* idempotent:** The server will receive the *second* request and charge the user *again*. **If the `/pay` endpoint *is* idempotent:** The server will recognize the second request as a duplicate and simply return the *original* result (e.g., 'Payment Successful') without processing it again. **How to implement it:** The client generates a unique **`Idempotency-Key`** (like a UUID) and sends it in the request header. The server stores a mapping of these keys (e.g., in Redis) to their responses. When a request comes in, the server checks if the key has been seen. If yes, it returns the saved response. If no, it processes the request, saves the response against the key, and then returns it. `GET`, `PUT`, and `DELETE` requests are often idempotent by definition. `POST` requests are *not* and must be explicitly designed to be.

Q41. What is the 'Thundering Herd' problem, and what are some strategies to mitigate it?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: The 'Thundering Herd' problem occurs when a large number of processes or threads, all waiting for the same event, are 'woken up' at the same time and all try to access the same resource. This sudden, massive spike in traffic can overwhelm the resource. **Classic Example (Cache Stampede):** 1. You have a very popular piece of data in your cache (e.g., the 'Top News' homepage) with a 60-second TTL. 2. At 10:00:00 AM, the cache key expires. 3. In the next second (10:00:01 AM), 10,000 user requests arrive for this data. 4. All 10,000 requests see a 'cache miss' and *all 10,000* try to run the *same* expensive query against your database to regenerate the data. This is a thundering herd that can crash your database. **Mitigation Strategies:** 1. **Cache Pre-fetching (Stale-while-revalidate):** When a request sees a *stale* (expired) cache item, it serves the stale data to the user *immediately* (which is fast) and then *asynchronously* triggers a *single* background job to refresh the cache. 2. **Locking (Mutex):** When a cache miss occurs, the *first* request to arrive will acquire a 'lock' (e.g., in Redis using `SETNX`). This request is now responsible for regenerating the data. All other requests that arrive in the meantime will *wait* for the lock to be released, and then read the data that the first request populated. 3. **Probabilistic Early Expiration:** Instead of a hard TTL, you set a 'soft' TTL. When a request sees an item nearing its 'soft' expiration, it has a small *chance* of being the one to trigger an asynchronous background refresh. This staggers the refresh operations over time.

Q42. Explain the 'CQRS' (Command Query Responsibility Segregation) pattern. What is its main benefit?

[HARD] | ***Category:*** *System Design & Architecture*

Answer: CQRS is an architectural pattern that *segregates* (separates) the part of your application that *writes* data (the 'Command' side) from the part that *reads* data (the 'Query' side). **Traditional Model:** In a standard application, you have *one* data model and *one* database (e.g., a 'User' model and a 'User' table) that is used for both writes (creating/updating a user) and reads (getting a user's profile). This model is simple but not optimized for either operation. **CQRS Model:** 1. **Command Side:** This side handles all create, update, and delete operations. It uses a write-optimized data model and database (e.g., an 'Event Sourcing' model). Its only job is to validate and execute commands, ensuring data consistency. It does not return data. 2. **Query Side:** This side handles all read operations. It uses one or more read-optimized, *denormalized* 'read models' or 'materialized views'. These read models are specifically tailored to the queries the UI needs to make, making them extremely fast (e.g., no `JOINs`). **How they sync:** The Query side is updated *asynchronously*. It listens to events published by the Command side (e.g., `USER_UPDATED`) and updates its own read models. **Main Benefit:** **Performance & Scalability.** You can scale the Command and Query sides *independently*. You can have 3 write servers and 50 read servers. The Query side's read models are highly denormalized, making queries extremely fast, which is perfect for complex UIs. It's often used with Event Sourcing.

Q43. What is the difference between a 'proxy' and a 'reverse proxy'? Provide a common example of each.

[MEDIUM] | ***Category:*** *System Design & Architecture*

Answer: This is a fundamental networking concept. The difference is in who the proxy is protecting and who it is acting on behalf of. **1. Forward Proxy (or 'Proxy'):** A forward proxy sits *in front of client machines* (e.g., on a corporate or school network) and acts on behalf of the **client**. When an employee tries to visit `google.com`, their browser request goes to the proxy *first*. The proxy then forwards the request to `google.com` on their behalf. **Use Cases:** * **Filtering/Security:** The company can use the proxy to block access to certain websites (like social media). * **Anonymity:** The outside world (e.g., `google.com`) sees the request as coming from the proxy's IP address, not the client's. **2. Reverse Proxy:** A reverse proxy sits *in front of server machines* and acts on behalf of the **server**. When a user tries to visit `MyWebApp.com`, their request hits the reverse proxy *first*. The reverse proxy then forwards the request to one of many backend application servers. The client has no idea the reverse proxy exists. **Use Cases (This is what Nginx and API Gateways are):** * **Load Balancing:** Distributing traffic across multiple backend servers. * **SSL Termination:** Handling the HTTPS encryption/decryption so the backend servers don't have to. * **Caching:** Caching static content. * **Security:** Hiding the internal network topology from the public.